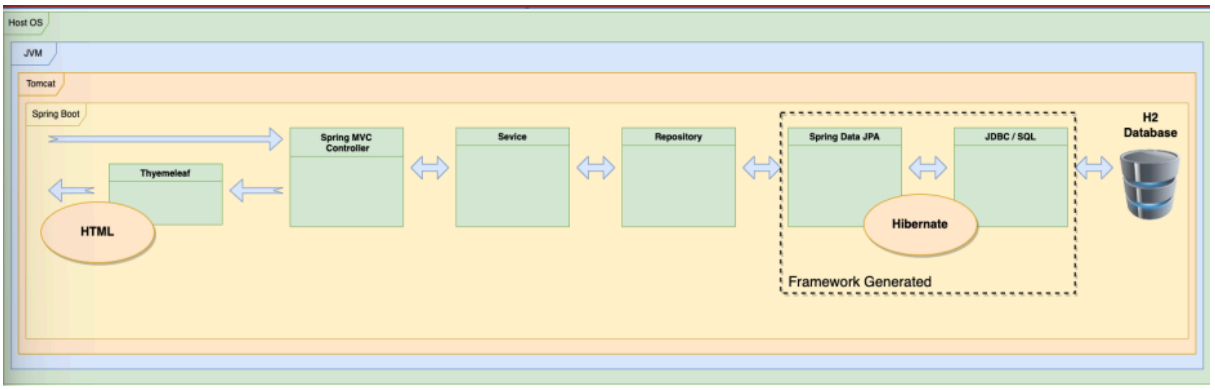
13. Overview of the Spring Boot Application We Will Build

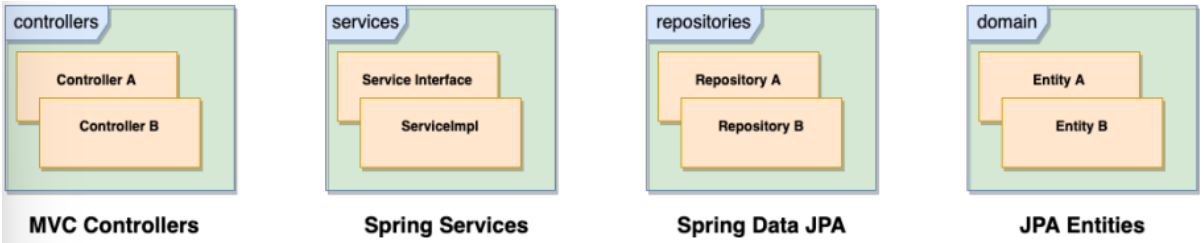
In this video, we are going to do an overview of the web app that we are going to be building in this section. So in this section we are going to build a very simple web based application. So what's going to happen is we are going to be talking via a web browser. Doesn't matter which browser you use, but we are going to be talking to a spring boot application that is going to be listening on port 8080. So normally the default is 80 here. Traditionally Java applications will start up on 8080. That's a long time standard for Java application servers. And Spring Boot does have an embedded Tomcat server. And it is going to by default. Listen on 8080. What happens is Tomcat listens on 8080 and then routes the request to the spring boot application. And being that it is a web browser, Spring Boot is going to be responding with HTML and everything is going to be handled by HTTP. So we're going to be following the standards on there. I just want to set the expectations in this section. We are going to be building a very basic web application for consumption by a web browser. So be following HTTP and also using HTML. Now the runtime environment that does get a little more interesting just to help you visualize what's going to be happening when we start it. You're going to have your host operating system when you start up the spring boot application, obviously that's going to be running within the JVM. Within the JVM. Tomcat will start up an embedded Tomcat instance, and within Tomcat, Spring Boot will be executing. So that will start up as an application within Tomcat. And what happens when a request comes in. That request is going to get routed to a spring MVC controller. We will have a service layer that we'll write that will handle that request. Or the controller actually handles the request, but the request gets passed to a service. When we are doing database operations, we will be using Spring Data JPA repositories. So we you'll see that we write the repository interface. Actually the implementation is provided by Spring Data JPA. So as we get over to the right hand side of the screen there, I do have a dotted box of framework generated that is going to be all generated for us. So we don't have to write any SQL code at all. And that will be handled by Spring Data JPA and the hibernate implementation. Then of course underneath the covers, hibernate is going to be utilizing JDBC and will be generating the SQL to talk to the H2 in-memory database. So the application that we are building is going to be talking to an in-

memory database. And that coordination with the database is going to be handled by hibernate which implements the JPA standard. Now once we get the data out of the database, that request is going to be passed back up the chain through the repository service layer. The spring MVC controller will provide data back to the Time life engine. And then that's going to generate the time life engine will generate the HTML response for the browser to consume. To help you visualize the components that we will be writing, we generally follow this design pattern. So we will create packages for controllers, services, repositories and domain. So within the controllers package we'll have spring MVC controllers. Services are going to be custom services. And the services would typically write to an interface and then create an implementation. Later in the course. You'll see how this becomes more important for dependency injection to be writing to an interface. So a lot of times you don't do controllers in interface because you don't have any need to be performing dependency injection on them. However, services that's very common to write multiple implementations of services. So we will follow that design pattern inside this course. And then repositories you'll see coming up in the course you provide an interface for the repository. And Spring Data JPA will generate the implementation. And then finally we will have a package called domain. And within that package we will create pojos plain old Java objects which are just objects with properties and getters and setters. And we will configure those to be JPA entities that JPA slash hibernate will use for persistence operations. So this is just to give you an overview of what we are doing in this section of the course. And coming up we will start implementing each piece of this. So we'll start off with the domain objects. Start mapping those up and then we will develop the rest of the application. By the end of this section we will have a web application that allows you to persist and retrieve data from the in-memory database.

Spring Web App Overview

- Web Based Application
- Browser Communicates via HTTP on port 8080
    - Normal default is 80
- Tomcat Server Listens for Requests on port 8080
- Tomcat Routes Requests to Spring Boot Application
- Spring Boot Responds with HTML via HTTP





Visão Geral da Aplicação Spring Boot que Vamos Construir

Neste vídeo, faremos uma visão geral da aplicação web que iremos construir nesta seção. Nesta etapa do curso, desenvolveremos uma aplicação web bastante simples. O que acontecerá é que nos comunicaremos por meio de um navegador web. Não importa qual navegador você utilize, pois estaremos nos comunicando com uma aplicação Spring Boot que ficará escutando na porta 8080.

Normalmente, o padrão para aplicações web é a porta 80. Tradicionalmente, aplicações Java iniciam na porta 8080. Esse é um padrão de longa data para servidores de aplicação Java. O Spring Boot possui um servidor Tomcat embarcado e, por padrão, ele ficará escutando na porta 8080. O que acontece é que o Tomcat escuta na porta 8080 e, em seguida, encaminha a requisição para a aplicação Spring Boot.

Como se trata de um navegador web, o Spring Boot responderá com HTML, e toda a comunicação será tratada via HTTP. Portanto, seguiremos os padrões

definidos para HTTP. Quero apenas alinhar as expectativas nesta seção: construiremos uma aplicação web bastante básica para consumo por um navegador, seguindo os padrões HTTP e utilizando HTML.

Agora, o ambiente de execução se torna um pouco mais interessante, apenas para ajudar você a visualizar o que acontecerá quando iniciarmos a aplicação. Você terá o sistema operacional host e, ao iniciar a aplicação Spring Boot, ela será executada dentro da JVM. Dentro da JVM, o Tomcat iniciará uma instância embarcada e, dentro do Tomcat, o Spring Boot será executado. Assim, a aplicação será iniciada como uma aplicação dentro do Tomcat.

Quando uma requisição chega, ela será encaminhada para um controller Spring MVC. Teremos uma camada de serviço que iremos implementar para lidar com essa requisição. Tecnicamente, o controller recebe a requisição, mas ela é delegada para a camada de serviço.

Quando realizarmos operações de banco de dados, utilizaremos repositórios do Spring Data JPA. Você verá que escreveremos apenas a interface do repositório; a implementação é fornecida automaticamente pelo Spring Data JPA. No lado direito do diagrama, há uma área pontilhada representando código gerado pelo framework, que será todo gerado automaticamente. Assim, não precisaremos escrever nenhum código SQL manualmente.

Esse processamento será tratado pelo Spring Data JPA com a implementação do Hibernate. Por baixo dos panos, o Hibernate utilizará JDBC e gerará o SQL necessário para se comunicar com o banco de dados H2 em memória. Portanto, a aplicação que construiremos se comunicará com um banco de dados em memória, e essa coordenação com o banco será gerenciada pelo Hibernate, que implementa o padrão JPA.

Após a recuperação dos dados do banco, a requisição retornará pela cadeia de chamadas, passando pelo repositório e pela camada de serviço. O controller Spring MVC fornecerá os dados para o mecanismo Thymeleaf, que então gerará a resposta HTML para o navegador consumir.

Para ajudar você a visualizar os componentes que iremos desenvolver, geralmente seguimos este padrão de projeto. Criaremos pacotes para **controllers**, **services**, **repositories** e **domain**.

No pacote **controllers**, teremos os controllers Spring MVC. No pacote **services**, teremos serviços customizados. Normalmente, os serviços são definidos por meio de interfaces e implementações. Mais adiante no curso,

você verá como isso se torna importante para injeção de dependências, já que escrever código baseado em interfaces é uma prática recomendada.

Geralmente, não criamos interfaces para controllers, pois não há necessidade de realizar injeção de dependências sobre eles. Já para serviços, é bastante comum ter múltiplas implementações, então seguiremos esse padrão ao longo do curso.

No pacote **repositories**, você verá que fornecemos apenas a interface do repositório, e o Spring Data JPA gera automaticamente a implementação. Por fim, teremos o pacote **domain**, onde criaremos os POJOs (*Plain Old Java Objects*), que são objetos simples contendo propriedades com getters e setters. Esses objetos serão configurados como entidades JPA, que o JPA/Hibernate utilizará para operações de persistência.

Essa explicação serve apenas para fornecer uma visão geral do que faremos nesta seção do curso. Em seguida, começaremos a implementar cada uma dessas partes. Iniciaremos pelos objetos de domínio, mapeando-os, e então desenvolveremos o restante da aplicação. Ao final desta seção, teremos uma aplicação web que permitirá persistir e recuperar dados de um banco de dados em memória.

Visão Geral da Aplicação Web Spring

- Aplicação baseada na Web
- O navegador se comunica via HTTP na porta 8080
  - O padrão normal é a porta 80
- O servidor Tomcat escuta requisições na porta 8080
- O Tomcat encaminha as requisições para a aplicação Spring Boot
- O Spring Boot responde com HTML via HTTP

Se quiser, posso **padronizar essas traduções para anotações de estudo**, **gerar um resumo técnico**, ou **adaptar o texto para slides ou documentação interna**.